

BLE 및 Slope UART 인터페이스 - Bluetooth기반 교통약자용 버스 전동문 및 전동경사로 소규모 통합 제어 시스템

```
1  # BLE UART / Slope UART 인터페이스 명세
2
3  ## 1. 개요
4
5  `ble_uart`와 `slope_uart`는 동일한 UART 프레임 구조와 동일한 송수신 방식을 사
6
7  공통점은 다음과 같다.
8
9  - 6바이트 고정 길이 프레임
10 - SOF(Start Of Frame) 2바이트 사용
11 - `DATA` / `ACK` 프레임 구분
12 - `seq` 기반 ACK 확인
13 - CRC-8 검증
14 - 중복 수신 프레임 재전달 방지
15 - `poll()` 기반 처리 구조
16
17 차이점은 거의 `cmd` 바이트의 의미뿐이다.
18
19 - `ble_uart`: 문(door) + 슬로프(slope) 요청을 함께 담음
20 - `slope_uart`: 슬로프 요청만 담음
21
22 ---
23
24 ## 2. 프레임 명세
25
26 모든 프레임은 아래와 같은 고정 6바이트 형식을 사용한다.
27
28 | Byte Index | 이름 | 설명 |
29 |---|---|---|
30 | 0 | SOF0 | `0xA5` |
31 | 1 | SOF1 | `0x5A` |
32 | 2 | TYPE | `DATA(0x01)` 또는 `ACK(0x02)` |
33 | 3 | SEQ | 프레임 시퀀스 번호 |
34 | 4 | CMD | 명령 바이트 |
35 | 5 | CRC | `TYPE, SEQ, CMD` 3바이트에 대한 CRC-8 |
36
37 ### 2.1 상수
38
39 ##### 공통 SOF / TYPE
40
41 - `SOF0 = 0xA5`
42 - `SOF1 = 0x5A`
43 - `FRAME_SIZE = 6`
44 - `TYPE_DATA = 0x01`
45 - `TYPE_ACK = 0x02`
46
47 ### 2.2 CRC
48
49 CRC는 `TYPE`, `SEQ`, `CMD` 총 3바이트에 대해 계산한다.
50
51 - 초기값: `0x00`
52 - 다항식: `0x07`
53 - ACK 프레임의 `CMD` 필드는 `0x00`
54
55 즉,
56
57 - DATA 프레임 CRC = CRC8(`TYPE_DATA`, `SEQ`, `CMD`)
58 - ACK 프레임 CRC = CRC8(`TYPE_ACK`, `SEQ`, `0x00`)
59
60 ---
61
```

```

62  ## 3. 명령(cmd) 바이트 레이아웃
63
64  ## 3.1 `ble_uart`
65
66  `ble_uart`는 door와 slope 요청을 하나의 1바이트에 담는다.
67
68  - `bits[1:0]` = door request
69  - `bits[3:2]` = slope request
70  - `bits[7:4]` = 사용하지 않음, 반드시 0
71
72  ### 요청 값
73
74  - `0x00` = NOP
75  - `0x01` = OPEN
76  - `0x02` = CLOSE
77
78  ### 예시
79
80  - door=open, slope=nop
81  - `cmd = bleUartMakeCmd(BLE_UART_REQ_OPEN, BLE_UART_REQ_NOP)`
82  - door=nop, slope=close
83  - `cmd = bleUartMakeCmd(BLE_UART_REQ_NOP, BLE_UART_REQ_CLOSE)`
84  - door=open, slope=close
85  - `cmd = bleUartMakeCmd(BLE_UART_REQ_OPEN, BLE_UART_REQ_CLOSE)`
86
87  ### 해석 함수
88
89  - `bleUartCmdGetDoor(cmd)`
90  - `bleUartCmdGetSlope(cmd)`
91  - `bleUartCmdIsValid(cmd)`
92
93  ---
94
95  ## 3.2 `slope_uart`
96
97  `slope_uart`는 슬로프 요청만 1바이트에 담는다.
98
99  - `bits[1:0]` = slope request
100 - `bits[7:2]` = 사용하지 않음, 반드시 0
101
102 ### 요청 값
103
104 - `0x00` = NONE
105 - `0x01` = OPEN
106 - `0x02` = CLOSE
107
108 ### 예시
109
110 - slope open
111 - `cmd = slopeUartMakeCmd(SLOPE_UART_REQ_OPEN)`
112 - slope close
113 - `cmd = slopeUartMakeCmd(SLOPE_UART_REQ_CLOSE)`
114
115 ### 해석 함수
116
117 - `slopeUartCmdGetSlope(cmd)`
118 - `slopeUartCmdIsValid(cmd)`
119
120 ---
121
122 ## 4. 송수신 구조
123
124 ## 4.1 전체 흐름
125
126 이 프로토콜은 기본적으로 아래 순서로 동작한다.
127
128 1. 송신 측이 `DATA` 프레임 전송

```

```

129 2. 수신 측이 프레임 파싱 및 CRC 검증
130 3. 정상 프레임이면 같은 `SEQ`로 `ACK` 전송
131 4. 수신 측은 **처음 본 `SEQ`** 인 경우에만 명령을 상위 로직에 전달
132 5. 송신 측은 해당 `ACK`를 받으면 전송 성공으로 처리
133 6. ACK가 없으면 timeout 이후 재전송
134 7. retry limit 초과 시 give-up 처리
135
136 ---
137
138 ## 4.2 수신 측 구조
139
140 수신 측은 보통 아래 3단계로 구성된다.
141
142 ### 1. Parser
143
144 바이트 스트림을 1바이트씩 넣어서 프레임 단위로 복원한다.
145
146 - `ParserInit(...)`
147 - `ParserFeed(..., byte, &decoded)`
148
149 `ParserFeed()`의 반환값 의미:
150
151 - `PARSE_NONE` : 아직 프레임 완성 아님
152 - `PARSE_OK` : 정상 프레임 완성
153 - `PARSE_ERROR` : CRC 오류 또는 형식 오류
154
155 ### 2. Receiver
156
157 정상 decode된 프레임을 받아서 다음을 처리한다.
158
159 - DATA 프레임인지 확인
160 - cmd 유효성 검사
161 - ACK 생성 여부 결정
162 - 중복 프레임인지 확인
163 - 상위 로직으로 명령 전달 여부 결정
164
165 반환 플래그:
166
167 - `RX_SEND_ACK`
168 - `RX_DELIVER`
169
170 ### 3. Application consume
171
172 실제로 명령을 적용하는 쪽에서는 보통 다음 패턴을 사용한다.
173
174 - 드라이버/컨텍스트 내부에 `pending_cmd` 저장
175 - `has_pending_cmd` 플래그 세움
176 - 메인 루프에서 `ConsumeCmd()` 호출
177
178 즉, ISR 또는 low-level RX 처리와 실제 제어 로직을 분리하는 구조다.
179
180 ---
181
182 ## 4.3 중복 프레임 처리
183
184 수신 측은 마지막으로 전달한 `seq`를 기억한다.
185
186 - 새로운 `seq`이면:
187   - ACK 전송
188   - 명령 전달
189 - 같은 `seq`가 다시 오면:
190   - ACK는 다시 전송
191   - 명령은 다시 전달하지 않음
192
193 이 구조 덕분에 송신 측 재전송이 발생해도 상위 로직에서 같은 명령이 중복 적용되지
194
195 ---

```

```

196
197 ## 4.4 송신 측 구조
198
199 송신 측은 `Sender` 상태를 내부적으로 유지한다.
200
201 관리하는 대표 상태는 다음과 같다.
202
203 - 현재 ACK 대기 중인지
204 - 현재 기다리는 `pending_seq`
205 - 재전송 횟수
206 - 다음 송신에 사용할 `next_seq`
207 - 마지막 전송 시각
208 - ACK timeout 값
209 - 재전송 제한 횟수
210 - 마지막으로 보낸 `pending_frame`
211
212 ### 송신 절차
213
214 1. `SenderStart()`로 DATA 프레임 생성
215 2. UART로 프레임 전송
216 3. ACK 수신 시 `SenderOnFrame()` 호출
217 4. ACK가 맞으면 busy 해제
218 5. 주기적으로 `SenderPoll()` 호출
219 6. timeout이면 resend 또는 give-up
220
221 ---
222
223 ## 5. 기본 timing 정책
224
225 두 프로토콜 모두 기본값은 다음과 같다.
226
227 - ACK timeout: `20 ms`
228 - retry limit: `5`
229
230 즉, 송신 후 일정 시간 안에 ACK가 오지 않으면 재전송하고, 제한 횟수를 넘으면 실패
231
232 ---
233
234 ## 6. 공개 함수 사용 방식
235
236 아래는 구현 세부가 아니라, 상위 로직이 어떻게 쓰면 되는지에 초점을 맞춘 사용법이다
237
238 ## 6.1 `ble_uart` 공통 함수
239
240 ### 프레임/명령 생성 및 해석
241
242 ```c
243 uint8_t cmd = bleUartMakeCmd(BLE_UART_REQ_OPEN, BLE_UART_REQ_CLOSE);
244 uint8_t door_req = bleUartCmdGetDoor(cmd);
245 uint8_t slope_req = bleUartCmdGetSlope(cmd);
246 ```
247
248 ### 파서 사용
249
250 ```c
251 BleUartParser parser;
252 BleUartDecodedFrame decoded;
253
254 bleUartParserInit(&parser);
255
256 int rc = bleUartParserFeed(&parser, rx_byte, &decoded);
257 if (rc == BLE_UART_PARSE_OK)
258 {
259     /* decoded.type / decoded.seq / decoded.cmd 사용 */
260 }

```

```

261  ``
262
263  ### 수신기 사용
264  ``
265  ``c
266  BleUartReceiver receiver;
267  BleUartFrame ack_frame;
268  uint8_t cmd_to_deliver;
269  uint8_t flags;
270
271  bleUartReceiverInit(&receiver);
272
273  flags = bleUartReceiverOnFrame(&receiver, &decoded, &ack_frame, &cmd_t
274
275  if ((flags & BLE_UART_RX_SEND_ACK) != 0u)
276  {
277      /* ack_frame UART 전송 */
278  }
279
280  if ((flags & BLE_UART_RX_DELIVER) != 0u)
281  {
282      /* cmd_to_deliver를 상위 로직에 전달 */
283  }
284  ``
285
286  ### 송신기 사용
287  ``
288  ``c
289  BleUartSender sender;
290  BleUartFrame tx_frame;
291
292  bleUartSenderInit(&sender, BLE_UART_DEFAULT_ACK_TIMEOUT_MS, BLE_UART_D
293
294  if (bleUartSenderStart(&sender, cmd, now_ms, &tx_frame) != 0)
295  {
296      /* tx_frame UART 전송 */
297  }
298  ``
299
300  ACK 수신 시:
301  ``
302  ``c
303  if (bleUartSenderOnFrame(&sender, &decoded) != 0)
304  {
305      /* 전송 성공 처리 */
306  }
307  ``
308
309  주기 poll 시:
310  ``
311  ``c
312  int poll_rc = bleUartSenderPoll(&sender, now_ms, &tx_frame);
313  if (poll_rc == BLE_UART_SENDER_POLL_RESEND)
314  {
315      /* tx_frame 재전송 */
316  }
317  else if (poll_rc == BLE_UART_SENDER_POLL_GIVEUP)
318  {
319      /* 실패 처리 */
320  }
321  ``
322

```

```

323 ---
324
325 ## 6.2 `ble_uart_tc375_lite` 사용 예시
326
327 `ble_uart_tc375_lite`는 **TC375에서 수신 전용 래퍼처럼 사용하는 형태**다.
328
329 ### 초기화
330
331 ```c
332 bleUartTc375Init();
333 ```
334
335 ### 메인 루프 poll
336
337 ```c
338 bleUartTc375Poll();
339 ```
340
341 ### 명령 소비
342
343 ```c
344 uint8_t cmd;
345
346 while (bleUartTc375ConsumeCmd(&cmd) != 0u)
347 {
348     uint8_t door_req = bleUartCmdGetDoor(cmd);
349     uint8_t slope_req = bleUartCmdGetSlope(cmd);
350
351     if (door_req == BLE_UART_REQ_OPEN)
352     {
353         /* door open 처리 */
354     }
355
356     if (slope_req == BLE_UART_REQ_CLOSE)
357     {
358         /* slope close 처리 */
359     }
360 }
361 ```
362
363 ### 특징
364
365 - 내부적으로 UART RX 바이트를 읽음
366 - 프레임 파싱
367 - ACK 자동 전송
368 - 유효한 명령이면 `pending_cmd` 저장
369 - 사용자는 `ConsumeCmd()`만 호출하면 됨
370
371 ---
372
373 ## 6.3 `slope_uart` 공통 함수
374
375 ### 명령 생성 및 해석
376
377 ```c
378 uint8_t cmd = slopeUartMakeCmd(SLOPE_UART_REQ_OPEN);
379 uint8_t slope_req = slopeUartCmdGetSlope(cmd);
380 ```
381
382 ### 파서 사용
383
384 ```c
385 SlopeUartParser parser;
386 SlopeUartDecodedFrame decoded;

```

```

387
388 slopeUartParserInit(&parser);
389
390 int rc = slopeUartParserFeed(&parser, rx_byte, &decoded);
391 if (rc == SLOPE_UART_PARSE_OK)
392 {
393     /* decoded.type / decoded.seq / decoded.cmd 사용 */
394 }
395 ...
396
397 ### 수신기 사용
398 ```c
399 SlopeUartReceiver receiver;
400 SlopeUartFrame ack_frame;
401 uint8_t cmd_to_deliver;
402 uint8_t flags;
403
404 slopeUartReceiverInit(&receiver);
405
406
407 flags = slopeUartReceiverOnFrame(&receiver, &decoded, &ack_frame, &cmd
408
409 if ((flags & SLOPE_UART_RX_SEND_ACK) != 0u)
410 {
411     /* ack_frame UART 전송 */
412 }
413
414 if ((flags & SLOPE_UART_RX_DELIVER) != 0u)
415 {
416     /* cmd_to_deliver를 상위 로직에 전달 */
417 }
418 ...
419
420 ### 송신기 사용
421 ```c
422 SlopeUartSender sender;
423 SlopeUartFrame tx_frame;
424 uint8_t cmd = slopeUartMakeCmd(SLOPE_UART_REQ_CLOSE);
425
426
427 slopeUartSenderInit(&sender, SLOPE_UART_DEFAULT_ACK_TIMEOUT_MS, SLOPE_
428
429 if (slopeUartSenderStart(&sender, cmd, now_ms, &tx_frame) != 0)
430 {
431     /* tx_frame UART 전송 */
432 }
433 ...
434
435 ACK 수신 시:
436 ```c
437 if (slopeUartSenderOnFrame(&sender, &decoded) != 0)
438 {
439     /* 전송 성공 처리 */
440 }
441 ...
442
443
444 주기 poll 시:
445 ```c
446 int poll_rc = slopeUartSenderPoll(&sender, now_ms, &tx_frame);
447

```

```

448 if (poll_rc == SLOPE_UART_SENDER_POLL_RESEND)
449 {
450     /* tx_frame 재전송 */
451 }
452 else if (poll_rc == SLOPE_UART_SENDER_POLL_GIVEUP)
453 {
454     /* 실패 처리 */
455 }
456 ...
457
458 ---
459
460 ## 6.4 `slope_uart_tc375_lite` 사용 예시
461
462 `slope_uart_tc375_lite`는 **TC375에서 송신 전용 래퍼처럼 사용하는 형태**다.
463
464 ### 초기화
465
466 ```c
467 slopeUartTc375TxInit();
468 ```
469
470 ### 원하는 명령 설정
471
472 ```c
473 slopeUartTc375RequestOpen();
474 /* 또는 */
475 slopeUartTc375RequestClose();
476 /* 또는 */
477 slopeUartTc375RequestNone();
478 ```
479
480 ### 메인 루프 poll
481
482 ```c
483 slopeUartTc375TxPoll();
484 ```
485
486 이 poll 내부에서 대략 다음이 수행된다.
487
488 - ACK 수신 처리
489 - timeout 기반 재전송 처리
490 - 현재 desired command를 아직 보내지 않았다면 송신 시작
491
492 ### 전송 성공 확인
493
494 ```c
495 uint8_t cmd;
496 if (slopeUartTc375ConsumeTxSuccess(&cmd) != 0u)
497 {
498     uint8_t slope_req = slopeUartCmdGetSlope(cmd);
499     /* 마지막 성공 명령 확인 */
500 }
501 ...
502
503 ### busy 확인
504
505 ```c
506 if (slopeUartTc375TxIsBusy()) != 0u)
507 {
508     /* 현재 ACK 대기 중 */
509 }
510 ...
511
512 ### 특징
513
514 - 사용자는 원하는 명령만 설정하면 됨

```



```

515 - 실제 프레임 생성, seq 관리, ACK 대기, 재전송은 내부 처리
516 - 같은 desired command를 중복으로 계속 보내지 않도록 제어함
517 ---
518 ---
519
520 ## 6.5 `slope_uart_stm32` 사용 예시
521
522 `slope_uart_stm32`는 **STM32에서 수신 래퍼**로 사용하는 구조다.
523
524 ### 초기화
525
526 ```c
527 SlopeUartStm32RxContext ctx;
528 slopeUartStm32RxInit(&ctx, &huart2);
529 ```
530
531 ### RX 바이트 입력
532
533 UART RX complete callback 등에서 1바이트씩 전달한다.
534
535 ```c
536 slopeUartStm32nRxByte(&ctx, rx_byte);
537 ```
538
539 ### 명령 소비
540
541 ```c
542 uint8_t cmd;
543 if (slopeUartStm32ConsumeCmd(&ctx, &cmd) != 0u)
544 {
545     uint8_t slope_req = slopeUartCmdGetSlope(cmd);
546
547     if (slope_req == SLOPE_UART_REQ_OPEN)
548     {
549         /* slope open 처리 */
550     }
551 }
552 ```
553
554 ### 특징
555
556 - 정상 DATA 수신 시 ACK 자동 송신
557 - 중복 프레임은 재전달하지 않음
558 - 상위 로직은 `ConsumeCmd()`로 명령만 꺼내서 사용
559 ---
560 ---
561
562 ## 7. 추천 사용 패턴
563
564 ## 7.1 수신 측
565
566 수신 전용 쪽은 아래 패턴이 가장 단순하다.
567
568 1. UART ISR 또는 callback에서 바이트 수신
569 2. parser에 1바이트씩 feed
570 3. decode 성공 시 receiver 처리
571 4. ACK 자동 송신
572 5. 유효 명령이면 pending queue 또는 pending flag에 저장
573 6. 메인 루프에서 `ConsumeCmd()` 호출하여 실제 제어
574
575 이 방식은 인터럽트 문맥에서 무거운 제어를 직접 하지 않고, 메인 루프에서 안정적으로
576
577 ---
578
579 ## 7.2 송신 측
580

```

```
581 송신 전용 쪽은 아래 패턴이 적합하다.
582
583 1. 상위 로직이 desired command 설정
584 2. `poll()`에서 sender가 idle이면 송신 시작
585 3. ACK를 기다림
586 4. ACK가 오면 성공 처리
587 5. timeout이면 재전송
588 6. retry 초과 시 실패 처리
589
590 즉, 요청은 event처럼 넣고, 실제 전송 상태 관리는 sender가 맡는 구조다.
```


BLE UART / Slope UART 인터페이스 명세

1. 개요

`ble_uart`와 `slope_uart`는 동일한 UART 프레임 구조와 동일한 송수신 방식을 사용하는 경량 명령 프로토콜이다.

공통점은 다음과 같다.

- 6바이트 고정 길이 프레임
- SOF(Start Of Frame) 2바이트 사용
- `DATA` / `ACK` 프레임 구분
- `seq` 기반 ACK 확인
- CRC-8 검증
- 중복 수신 프레임 재전달 방지
- `poll()` 기반 처리 구조

차이점은 거의 `cmd` 바이트의 의미뿐이다.

- `ble_uart`: 문(door) + 슬로프(slope) 요청을 함께 담음
- `slope_uart`: 슬로프 요청만 담음

2. 프레임 명세

모든 프레임은 아래와 같은 고정 6바이트 형식을 사용한다.

Byte Index	이름	설명
0	SOF0	<code>0xA5</code>
1	SOF1	<code>0x5A</code>
2	TYPE	<code>DATA(0x01)</code> 또는 <code>ACK(0x02)</code>
3	SEQ	프레임 시퀀스 번호
4	CMD	명령 바이트
5	CRC	<code>TYPE</code> , <code>SEQ</code> , <code>CMD</code> 3바이트에 대한 CRC-8

2.1 상수

공통 `SOF` / `TYPE`

- `SOF0` = `0xA5`
- `SOF1` = `0x5A`
- `FRAME_SIZE` = 6
- `TYPE_DATA` = `0x01`
- `TYPE_ACK` = `0x02`

2.2 CRC

CRC는 `TYPE`, `SEQ`, `CMD` 총 3바이트에 대해 계산한다.

- 초기값: `0x00`
- 다항식: `0x07`
- ACK 프레임의 `CMD` 필드는 `0x00`

즉,

- DATA 프레임 CRC = `CRC8(TYPE_DATA, SEQ, CMD)`
- ACK 프레임 CRC = `CRC8(TYPE_ACK, SEQ, 0x00)`

3. 명령(cmd) 바이트 레이아웃

3.1 `ble_uart`

`ble_uart`는 door와 slope 요청을 하나의 1바이트에 담는다.

- `bits[1:0]` = door request
- `bits[3:2]` = slope request
- `bits[7:4]` = 사용하지 않음, 반드시 0

- 0x00 = NOP
- 0x01 = OPEN
- 0x02 = CLOSE

예시

- door=open, slope=nop
 - cmd = bleUartMakeCmd(BLE_UART_REQ_OPEN, BLE_UART_REQ_NOP)
- door=nop, slope=close
 - cmd = bleUartMakeCmd(BLE_UART_REQ_NOP, BLE_UART_REQ_CLOSE)
- door=open, slope=close
 - cmd = bleUartMakeCmd(BLE_UART_REQ_OPEN, BLE_UART_REQ_CLOSE)

해석 함수

- bleUartCmdGetDoor(cmd)
- bleUartCmdGetSlope(cmd)
- bleUartCmdIsValid(cmd)

3.2 slope_uart

slope_uart는 슬로프 요청만 1바이트에 담는다.

- bits[1:0] = slope request
- bits[7:2] = 사용하지 않음, 반드시 0

요청 값

- 0x00 = NONE
- 0x01 = OPEN
- 0x02 = CLOSE

예시

- slope open
 - cmd = slopeUartMakeCmd(SLOPE_UART_REQ_OPEN)
- slope close
 - cmd = slopeUartMakeCmd(SLOPE_UART_REQ_CLOSE)

해석 함수

- slopeUartCmdGetSlope(cmd)
- slopeUartCmdIsValid(cmd)

4. 송수신 구조

4.1 전체 흐름

이 프로토콜은 기본적으로 아래 순서로 동작한다.

1. 송신 측이 DATA 프레임 전송
2. 수신 측이 프레임 파싱 및 CRC 검증
3. 정상 프레임이면 같은 SEQ로 ACK 전송
4. 수신 측은 처음 본 SEQ 인 경우에만 명령을 상위 로직에 전달
5. 송신 측은 해당 ACK를 받으면 전송 성공으로 처리
6. ACK가 없으면 timeout 이후 재전송
7. retry limit 초과 시 give-up 처리

4.2 수신 측 구조

수신 측은 보통 아래 3단계로 구성된다.

1. Parser

바이트 스트림을 1바이트씩 넣어서 프레임 단위로 복원한다.

- ParserInit(...)
- ParserFeed(..., byte, &decoded)

ParserFeed()의 반환값 의미:

- `PARSE_NONE` : 아직 프레임 완성 아님
- `PARSE_OK` : 정상 프레임 완성
- `PARSE_ERROR` : CRC 오류 또는 형식 오류

2. Receiver

정상 decode된 프레임을 받아서 다음을 처리한다.

- DATA 프레임인지 확인
- cmd 유효성 검사
- ACK 생성 여부 결정
- 중복 프레임인지 확인
- 상위 로직으로 명령 전달 여부 결정

반환 플래그:

- `RX_SEND_ACK`
- `RX_DELIVER`

3. Application consume

실제로 명령을 적용하는 쪽에서는 보통 다음 패턴을 사용한다.

- 드라이버/컨텍스트 내부에 `pending_cmd` 저장
- `has_pending_cmd` 플래그 세움
- 메인 루프에서 `ConsumeCmd()` 호출

즉, ISR 또는 low-level RX 처리와 실제 제어 로직을 분리하는 구조다.

4.3 중복 프레임 처리

수신 측은 마지막으로 전달한 `seq`를 기억한다.

- 새로운 `seq`이면:
 - ACK 전송
 - 명령 전달
- 같은 `seq`가 다시 오면:
 - ACK는 다시 전송
 - 명령은 다시 전달하지 않음

이 구조 덕분에 송신 측 재전송이 발생해도 상위 로직에서 같은 명령이 중복 적용되지 않는다.

4.4 송신 측 구조

송신 측은 `Sender` 상태를 내부적으로 유지한다.

관리하는 대표 상태는 다음과 같다.

- 현재 ACK 대기 중인지
- 현재 기다리는 `pending_seq`
- 재전송 횟수
- 다음 송신에 사용할 `next_seq`
- 마지막 전송 시각
- ACK timeout 값
- 재전송 제한 횟수
- 마지막으로 보낸 `pending_frame`

송신 절차

1. `SenderStart()`로 DATA 프레임 생성
2. UART로 프레임 전송
3. ACK 수신 시 `SenderOnFrame()` 호출
4. ACK가 맞으면 busy 해제
5. 주기적으로 `SenderPoll()` 호출
6. timeout이면 resend 또는 give-up

5. 기본 timing 정책

두 프로토콜 모두 기본값은 다음과 같다.

- ACK timeout: 20 ms

- retry limit: 5

즉, 송신 후 일정 시간 안에 ACK가 오지 않으면 재전송하고, 제한 횟수를 넘으면 실패로 본다.

6. 공개 함수 사용 방식

아래는 구현 세부가 아니라, 상위 로직이 어떻게 쓰면 되는지에 초점을 맞춘 사용법이다.

6.1 ble_uart 공통 함수

프레임/명령 생성 및 해석

```
uint8_t cmd = bleUartMakeCmd(BLE_UART_REQ_OPEN, BLE_UART_REQ_CLOSE);
uint8_t door_req = bleUartCmdGetDoor(cmd);
uint8_t slope_req = bleUartCmdGetSlope(cmd);
```

파서 사용

```
BleUartParser parser;
BleUartDecodedFrame decoded;

bleUartParserInit(&parser);

int rc = bleUartParserFeed(&parser, rx_byte, &decoded);
if (rc == BLE_UART_PARSE_OK)
{
    /* decoded.type / decoded.seq / decoded.cmd 사용 */
}
```

수신기 사용

```
BleUartReceiver receiver;
BleUartFrame ack_frame;
uint8_t cmd_to_deliver;
uint8_t flags;

bleUartReceiverInit(&receiver);

flags = bleUartReceiverOnFrame(&receiver, &decoded, &ack_frame, &cmd_to_deliver);

if ((flags & BLE_UART_RX_SEND_ACK) != 0u)
{
    /* ack_frame UART 전송 */
}

if ((flags & BLE_UART_RX_DELIVER) != 0u)
{
    /* cmd_to_deliver를 상위 로직에 전달 */
}
```

송신기 사용

```
BleUartSender sender;
BleUartFrame tx_frame;

bleUartSenderInit(&sender, BLE_UART_DEFAULT_ACK_TIMEOUT_MS, BLE_UART_DEFAULT_RETRY_LIMIT);

if (bleUartSenderStart(&sender, cmd, now_ms, &tx_frame) != 0)
{
    /* tx_frame UART 전송 */
}
```

ACK 수신 시:

```
if (bleUartSenderOnFrame(&sender, &decoded) != 0)
{
    // ...
}
```

```

    /* 전송 성공 처리 */
}

```

주기 poll 시:

```

int poll_rc = bleUartSenderPoll(&sender, now_ms, &tx_frame);
if (poll_rc == BLE_UART_SENDER_POLL_RESEND)
{
    /* tx_frame 재전송 */
}
else if (poll_rc == BLE_UART_SENDER_POLL_GIVEUP)
{
    /* 실패 처리 */
}

```

6.2 ble_uart_tc375_lite 사용 예시

ble_uart_tc375_lite는 TC375에서 수신 전용 래퍼처럼 사용하는 형태다.

초기화

```
bleUartTc375Init();
```

메인 루프 poll

```
bleUartTc375Poll();
```

명령 소비

```

uint8_t cmd;

while (bleUartTc375ConsumeCmd(&cmd) != 0u)
{
    uint8_t door_req = bleUartCmdGetDoor(cmd);
    uint8_t slope_req = bleUartCmdGetSlope(cmd);

    if (door_req == BLE_UART_REQ_OPEN)
    {
        /* door open 처리 */
    }

    if (slope_req == BLE_UART_REQ_CLOSE)
    {
        /* slope close 처리 */
    }
}

```

특징

- 내부적으로 UART RX 바이트를 읽음
- 프레임 파싱
- ACK 자동 전송
- 유효한 명령이면 pending_cmd 저장
- 사용자는 ConsumeCmd()만 호출하면 됨

6.3 slope_uart 공통 함수

명령 생성 및 해석

```

uint8_t cmd = slopeUartMakeCmd(SLOPE_UART_REQ_OPEN);
uint8_t slope_req = slopeUartCmdGetSlope(cmd);

```


파싱 사용

```
SlopeUartParser parser;
SlopeUartDecodedFrame decoded;

slopeUartParserInit(&parser);

int rc = slopeUartParserFeed(&parser, rx_byte, &decoded);
if (rc == SLOPE_UART_PARSE_OK)
{
    /* decoded.type / decoded.seq / decoded.cmd 사용 */
}
```

수신기 사용

```
SlopeUartReceiver receiver;
SlopeUartFrame ack_frame;
uint8_t cmd_to_deliver;
uint8_t flags;

slopeUartReceiverInit(&receiver);

flags = slopeUartReceiverOnFrame(&receiver, &decoded, &ack_frame, &cmd_to_deliver);

if ((flags & SLOPE_UART_RX_SEND_ACK) != 0u)
{
    /* ack_frame UART 전송 */
}

if ((flags & SLOPE_UART_RX_DELIVER) != 0u)
{
    /* cmd_to_deliver를 상위 로직에 전달 */
}
```

송신기 사용

```
SlopeUartSender sender;
SlopeUartFrame tx_frame;
uint8_t cmd = slopeUartMakeCmd(SLOPE_UART_REQ_CLOSE);

slopeUartSenderInit(&sender, SLOPE_UART_DEFAULT_ACK_TIMEOUT_MS,
SLOPE_UART_DEFAULT_RETRY_LIMIT);

if (slopeUartSenderStart(&sender, cmd, now_ms, &tx_frame) != 0)
{
    /* tx_frame UART 전송 */
}
```

ACK 수신 시:

```
if (slopeUartSenderOnFrame(&sender, &decoded) != 0)
{
    /* 전송 성공 처리 */
}
```

주기 poll 시:

```
int poll_rc = slopeUartSenderPoll(&sender, now_ms, &tx_frame);
if (poll_rc == SLOPE_UART_SENDER_POLL_RESEND)
{
    /* tx_frame 재전송 */
}
else if (poll_rc == SLOPE_UART_SENDER_POLL_GIVEUP)
{
    /* 실패 처리 */
}
```

6.4 slope_uart_tc375_lite 사용 예시

slope_uart_tc375_lite는 TC375에서 송신 전용 래퍼처럼 사용하는 형태다.

초기화

```
slopeUartTc375TxInit();
```

원하는 명령 설정

```
slopeUartTc375RequestOpen();  
/* 또는 */  
slopeUartTc375RequestClose();  
/* 또는 */  
slopeUartTc375RequestNone();
```

메인 루프 poll

```
slopeUartTc375TxPoll();
```

이 poll 내부에서 대략 다음이 수행된다.

- ACK 수신 처리
- timeout 기반 재전송 처리
- 현재 desired command를 아직 보내지 않았다면 송신 시작

전송 성공 확인

```
uint8_t cmd;  
if (slopeUartTc375ConsumeTxSuccess(&cmd) != 0u)  
{  
    uint8_t slope_req = slopeUartCmdGetSlope(cmd);  
    /* 마지막 성공 명령 확인 */  
}
```

busy 확인

```
if (slopeUartTc375TxIsBusy() != 0u)  
{  
    /* 현재 ACK 대기 중 */  
}
```

특징

